Mate Helper

Interactive Desktop Pet with AI

Architecture & User Guide

Animated desktop companion with AI conversations,
speech-to-text, text-to-speech, accessibility features,
Libras sign language translation, and extensible character models.

English Edition

Table of Contents

1 Introduction

Mate Helper is a Linux desktop application that brings an animated, AI-powered virtual pet to your screen. The pet character walks, dances, reacts emotionally, and interacts with you through speech bubbles, a text chat window, and voice commands. It can converse using multiple AI providers, execute real computer commands, describe what is happening on your screen, transcribe desktop audio, set alarms, and speak aloud with synthetic voices.

Designed for both utility and companionship, Mate Helper serves as a friendly desktop assistant that can help with everyday tasks while providing a charming, animated presence. It supports multiple character models with different personalities and languages, and includes accessibility features for users who benefit from screen reading and audio transcription.

Key Features

- Animated character with five emotional moods (normal, happy, sad, angry, dancing)
- Multi-provider AI chat: Ollama (local), Groq Cloud, Google Gemini, HuggingFace Inference API, or offline phrase library
- Speech-to-text via microphone (push-to-talk or continuous listening modes)
- Text-to-speech with three providers: Fish Audio, Edge TTS, and pyttsx3
- Automatic screen reading - periodic screenshots described by AI
- Automatic desktop audio transcription - captures and transcribes system audio
- Libras (Brazilian Sign Language) translation with colored sign cards in speech bubbles
- Seven AI-executable tools: open URLs, take screenshots, capture audio, read/write files, list directories, run commands, query system info
- Alarm clock with custom MP3 ringtones
- Extensible character model system with built-in templates
- Multi-language support: Portuguese, English, and Japanese UI translations
- Push-to-talk global shortcut for hands-free voice input from any application

Technology Stack

- Python 3.10+ with GTK 3, Cairo, Pango, and Pycairo for rendering
- Pillow for image processing and GIF animation support
- WebRTC VAD for voice activity detection on microphone input
- PulseAudio (parec) for audio capture; ffmpeg for audio format conversion
- Multiple AI API integrations: Ollama REST API, Groq SDK, Google Generative AI, HuggingFace Inference API
- Whisper (via Groq) for speech-to-text transcription
- pynput for global keyboard shortcut listening

Intended Audience

This guide is intended for users who want to understand how Mate Helper works under the hood, configure it for their needs, or create custom character models. It provides architectural descriptions of each subsystem without revealing proprietary source code.

2 Getting Started

System Requirements

- Linux operating system with X11 or Wayland display server
- Python 3.10 or newer
- GTK 3.0 runtime libraries with Cairo and Pango support
- PulseAudio sound server (for microphone and desktop audio capture)
- ffmpeg (for audio format conversion)
- Internet connection (for cloud AI providers and TTS services); Ollama mode works offline
- Optional: Ollama for fully local AI inference

Quick Start

1. Install system dependencies: GTK 3, Cairo, Pango, PulseAudio, ffmpeg, and Python 3 development headers.

2. Install Python packages: PyGObject, Pycairo, Pillow, requests, and any desired AI provider SDKs.

3. Clone or copy the application to your system.

4. Run the application: python3 desktop_pet/main.py

5. Right-click the pet to open the context menu and configure: AI provider, language, character model, and API keys.

6. Double-click the pet to open the chat window and start conversing.

First-Time Configuration

On first launch, the application creates a configuration file at ~/.config/mate-helper/config.json with sensible defaults. The pet appears in the center of your screen with a default character model. From the context menu, you can:

- Select an AI provider and enter API keys (for cloud services)
- Choose a character model and language
- Configure microphone settings for voice input
- Enable or disable TTS (text-to-speech)
- Set up alarms
- Configure accessibility automation (screen reading, audio transcription)
- Adjust window scale and speech bubble position

NOTE
For cloud AI providers (Groq, Gemini, HuggingFace), you will need to obtain API keys from their respective websites. The Ollama provider is fully local and requires no API key. The "Phrases" provider works entirely offline using the character's built-in phrase library.

3 System Architecture

Mate Helper follows a modular architecture where each subsystem handles a specific concern. The main application window orchestrates all components through a timer-based event loop, signal system, and shared configuration module. The following sections describe each layer of the architecture.

3.1 Module Overview

Module	Responsibility
app.py	Main window, event loop, speech queue, timer management, UI menus
character.py	Sprite loading, mood management, frame animation, Cairo rendering
ai.py	AI provider abstraction, API calls, transcription, Ollama lifecycle
chat.py	Chat window, conversation history, tool execution, mood detection
config.py	JSON config persistence, defaults, migrations, validation
tools.py	Screenshot, audio capture, file operations, command execution
tts.py	Text-to-speech provider abstraction, audio playback
libras.py	Libras phrase dictionary, grammar rules, translation engine
log.py	Timestamped logging to stderr
models/	Model proxy, model discovery, character definitions and strings

3.2 Event Loop and Scheduling

The main window runs several concurrent timing mechanisms. A GTK timeout (125 ms) drives the character animation loop. Additional timers manage periodic tasks such as random speech, screen reading, audio transcription, alarm checking, and continuous microphone listening. Each timer runs on the GTK main loop thread for UI updates, while heavy operations (AI queries, audio capture, file I/O) execute in separate daemon threads to keep the interface responsive.

3.3 Configuration Subsystem

All user settings are stored in a single JSON file at ~/.config/mate-helper/config.json. The configuration module provides load and save functions that merge saved settings with a comprehensive set of defaults. On load, the system validates provider names and bubble side settings against allowed values, and applies automatic migrations for older config formats. Settings are accessible from every module via the config module's dictionary.

NOTE
Changes made through the context menu are saved immediately to the config file. The application does not require a restart for most configuration changes - character model and language changes take effect on the next interaction.

3.4 Data Flow Overview

User interactions flow through the application as follows:

Mouse events (drag, double-click, right-click) are handled by the main window. Double-clicking opens the chat window. Right-clicking shows a context menu for configuration. Text entered in the chat window first passes through a keyword-based tool detection engine. If a tool command is recognized (e.g., "take a screenshot"), it is executed immediately and the result may be sent to the AI for natural-language commentary. If no tool matches, the message is sent directly to the configured AI

provider. AI responses are displayed as speech bubbles and optionally spoken via TTS.

Periodic accessibility tasks follow a separate flow: a timer captures a screenshot or audio clip, sends it to the AI for description or commentary, and displays the result as a speech bubble. The Libras translation system intercepts all speech bubble text when enabled, converting it to written Libras gloss notation before display.

4 Character System

The character system is responsible for loading, animating, and rendering the pet on screen. It supports multiple character models, each with their own visual appearance, personality, voice configuration, and language.

4.1 Rendering Pipeline

Characters are rendered on a GTK DrawingArea using Cairo graphics. The rendering pipeline processes each frame through these stages:

- The canvas is cleared with a fully transparent background (RGBA 0,0,0,0).
- The character's current mood and animation frame determine which sprite to draw.
- Sprites are scaled by the configured window scale factor (2x to 6x) for crisp pixel-art rendering.
- If the character is speaking, the "speaking" variant of the current mood sprite is used.
- The speech bubble is drawn on top of or beside the character, with auto-positioning logic.
- A Pango text layout engine renders speech text with word-wrap and custom fonts.
- When Libras mode is active, colored sign cards are drawn below the speech text.

4.2 Moods and Animation

Characters have five emotional moods, each with its own sprite set:

Mood	Description	Trigger
Normal	Idle/neutral state	Default state; after any temporary mood expires
Happy	Positive, cheerful	Praise, compliments, positive chat interactions
Sad	Down or upset	Insults, negative language, anger from user
Angry	Frustrated or annoyed	Strong negative language or hostility
Dancing	Celebratory animation	Alarm ringing, special events

Each mood sprite can have two variants: a static/idle version and a "speaking" version (with an open mouth or altered expression). The system automatically switches to the speaking variant when the pet is showing a speech bubble.

4.3 Sprite Formats

Two sprite formats are supported:

- Static PNG sprite sheets: A single PNG image containing all animation frames arranged horizontally. The system detects individual frame boundaries by scanning for alpha-channel transitions in the image. Each frame is assumed to be 32 pixels wide by default.
- Animated GIFs: The system uses Pillow to extract individual frames from GIF files along with their per-frame delay values. Frames are assembled into a horizontal strip internally and animated using the GIF's timing information.

Sprites are stored in a sprites/ subdirectory within each model's folder. Each mood requires a separate file (e.g., Default.png, Happy.png, Sad.png, Angry.png, Dancing.gif).

4.4 Built-in Characters

The application ships with two character types, each available in three languages:

Character	Personality	Languages

Kasane Teto	Energetic, playful, caring UTAUloid vocaloid. Uses emotes, slang, and short, lively replies.	Portuguese, English, Japanese
Computer / PC	Direct, efficient, technical assistant. Formal and straight-to-	Portuguese, English, Japanese

Each language variant has localized UI strings, culturally appropriate phrase libraries, and region-specific TTS voice configurations. Characters can be switched at runtime from the context menu without restarting the application.

4.5 Language System

The application supports full internationalization. Each character model includes a strings module containing translations for approximately 160 UI labels, messages, and dialog texts across Portuguese, English, and Japanese. The active language is selected from the context menu and persists across sessions. Character models are loaded based on the combination of the selected character and language.

NOTE
The character model system is designed for extensibility. Users can create custom characters by following the template provided in the docs/custom_model/ directory. A complete guide to custom model creation is provided in Chapter 10.

5 Artificial Intelligence

The AI subsystem provides a unified interface to multiple language model providers, enabling the pet to hold natural conversations, analyze visual content, transcribe audio, and execute real computer commands through an extensible tool framework.

5.1 Provider Architecture

All AI providers share a common interface: they receive a list of conversation messages (including system prompt, user messages, and optional image data) and return a text response. The system prompt, defined by the character model, establishes the pet's personality, speech patterns, and behavioral constraints.

The message builder automatically injects the user's name and biography (configured in the profile settings) into the conversation context. When tool permissions are enabled, a special tool-use instruction block is appended to the system prompt, teaching the AI how to issue structured tool commands.

5.2 Provider Details

Ollama (Local Inference)

Ollama runs large language models entirely on the local machine, requiring no internet connection or API keys. The application automatically manages the Ollama process: it checks if Ollama is running on startup, starts it if needed (via ollama serve), and terminates it on application exit. The system selects the largest available model from the user's local Ollama library, falling back to smaller models as needed.

This provider is ideal for users who prioritize privacy, want offline operation, or wish to avoid API usage costs. Response quality depends on the local model's capabilities and system resources.

Groq Cloud

Groq provides high-speed inference through their cloud API, using custom hardware accelerators for extremely low latency. The implementation uses the Llama 3.3 70B model for text conversations and Llama 4 Scout 17B for vision tasks (screenshot analysis). Audio transcription uses Groq's Whisper large-v3-turbo endpoint.

Groq offers a generous free tier, making it a good default choice for the 'Auto' option.fallback chain.

Google Gemini

The Gemini integration uses Google's Gemini 2.5 Flash model (with fallbacks to 2.0 Flash and 1.5 Flash). It supports multimodal input, including images for screenshot analysis. The system performs a DNS resolution check before each API call to verify internet connectivity and provides descriptive error messages on failure.

HuggingFace Inference API

HuggingFace provides community-hosted models through their Inference API. The implementation uses the Arch Router 1.5B model, a lightweight instruction-tuned model suitable for simple conversations and tool commands.

Offline Phrases

When no AI provider is configured or all cloud providers fail, the system falls back to the character model's built-in phrase library. This library contains categorized responses for common conversation scenarios (greetings, how-are-you, thanks, goodbyes, etc.) and includes a keyword-matching system that selects contextually appropriate responses.

5.3 Auto-Fallback Chain

The "Auto" provider mode implements a resilience chain: when a request fails with one provider (due to network errors, API limits, or authentication issues), the system automatically tries the next provider in order. The fallback order is:

Groq Cloud -> Google Gemini -> HuggingFace -> Offline Phrases

This ensures the pet always responds, even when individual cloud services are unavailable. The fallback is transparent to the user, who only sees the final response.

5.4 Vision Support

Two AI providers support image inputs for screenshot analysis: Groq (with Llama 4 Scout) and Gemini (native multimodal support). When an accessibility task triggers a screenshot capture, the image is base64-encoded and sent alongside a descriptive prompt from the character model. The AI then describes what it sees in the character's personality style.

5.5 Tool-Augmented AI

One of the system's most powerful features is its tool-augmented AI capability. The AI is instructed via its system prompt that it can issue structured commands using a TOOL: prefix. When the chat system detects this prefix in the AI's response, it intercepts the command, executes it, and optionally sends the result back to the AI for a natural-language summary.

The AI can execute these tool categories:

Tool	Description	Example
Open URL	Opens a website in the default browser	Open youtube.com
Screenshot	Captures and optionally analyzes the scree	What's on my screen?
Audio Capture	Records and transcribes desktop audio	What song is playing?
List Files	Shows directory contents	What's in my Downloads?
Read File	Displays file contents	Read my notes.txt
Write File	Creates or overwrites a file	Save this as todo.txt
Run Command	Executes a bash command (with safety filters)	Check system uptime

Each tool has built-in safety mechanisms: dangerous shell commands are blocked by a blacklist pattern matcher, file operations are size-limited, and all tool execution is gated by individual permission toggles in the settings.

6 Speech & Audio

The speech and audio subsystem provides bidirectional voice interaction: speech-to-text (STT) converts your voice into text for chat input, while text-to-speech (TTS) makes the pet speak aloud. Both systems support multiple backends with automatic fallback.

6.1 Speech-to-Text (STT)

Microphone Capture Modes

Two microphone input modes are available:

- Push-to-Talk (Hold): Press and hold a configurable key or the microphone button in the chat window. Release to stop recording and begin transcription. This mode is ideal for controlling exactly when the microphone is active.
- Toggle (Continuous): Click to start recording, click again to stop (with a 5-second auto-stop timeout). The system also supports a continuous listening mode that periodically captures audio every 8 seconds, automatically detecting whether speech is present.
- Global Push-to-Talk: A Unix socket server enables push-to-talk from any application using a configurable global shortcut (default: Win+V).

Voice Activity Detection

To prevent noise from being sent to the transcription engine, the system employs WebRTC Voice Activity Detection (VAD). Captured audio is divided into 30-millisecond frames, and each frame is analyzed by the VAD engine. If less than 5% of frames contain speech, the audio is discarded as noise. This effectively filters out ambient sounds like fans, keyboard typing, and background music while reliably capturing single-word utterances.

Transcription Engine

All transcription is handled by Groq's Whisper large-v3-turbo endpoint, which provides fast and accurate speech recognition primarily for Portuguese but supporting multiple languages. Audio is captured as 16-bit PCM at 16 kHz mono via PulseAudio, converted to WAV format, and sent to the Whisper API for transcription.

6.2 Text-to-Speech (TTS)

The TTS system makes the pet speak aloud whenever it shows a speech bubble. Three providers are available, each with different quality and infrastructure trade-offs.

Provider	Quality	Requires	Best For
Fish Audio	Highest	API key, internet	Natural voices, character-specific voices
Edge TTS	High	Internet	Free high-quality voices, multiple languages
pyttsx3	Low-Medium	espeak installed	Offline use, no API needed

Fish Audio

Fish Audio provides high-quality neural TTS via cloud API. The user must obtain an API key from fish.audio and configure it through the settings menu. Character models can specify a default voice ID, and users can override it with a custom voice. Fish Audio supports realistic voice cloning and produces the most natural-sounding speech.

Edge TTS

Edge TTS uses Microsoft's Edge browser TTS service to generate speech. It offers high-quality voices in many languages without requiring an API key. Each character model can configure a specific Edge TTS voice (e.g., pt-BR-FranciscaNeural for Portuguese Teto). The implementation includes a 15-second timeout to handle slow network conditions.

pyttsx3

pyttsx3 provides offline TTS by wrapping espeak on Linux systems. While voice quality is lower than cloud alternatives, it works without internet access and requires no API keys or configuration. This is the ultimate fallback when cloud services are unavailable.

Auto-Fallback and Device Selection

When the TTS provider is set to "Auto", the system tries providers in order: Fish Audio first (if an API key is configured), then Edge TTS, and finally pyttsx3. Audio playback supports PulseAudio sink selection, allowing users to choose which audio output device receives the pet's voice.

7 Accessibility

Mate Helper includes several accessibility features designed to assist users with visual or hearing impairments, as well as users who prefer multimodal interaction.

7.1 Screen Reading Automation

When enabled, the system automatically captures screenshots at configurable intervals and sends them to the AI for visual description. The AI describes what it sees in the character's personality style. This can operate in two modes:

- Exact interval: A screenshot is taken at a fixed interval (e.g., every 60 seconds).
- Random interval: Screenshots are taken at random intervals between a configurable minimum and maximum.

This feature provides ambient awareness of on-screen activity for users who may have difficulty seeing screen content, or who want the pet to comment on what they are doing.

7.2 Desktop Audio Transcription

The system can periodically capture desktop audio (system output, including music, videos, and notifications) and send it to Whisper for transcription. The transcribed text is then sent to the AI for commentary. Like screen reading, this supports both exact and random interval modes.

Audio is captured from the PulseAudio monitor source, which records whatever audio is being played through the system's speakers. Captures are limited to 8 seconds to avoid excessive processing.

7.3 Libras (Brazilian Sign Language) Translation

The Libras translation module converts Portuguese speech bubble text into written Libras gloss notation, making the pet's speech accessible to Brazilian deaf users who read Libras. The translation system operates at three levels:

- Phrase dictionary: Approximately 40 common phrases (greetings, how-are-you, thanks, goodbyes, alarm messages, name introductions) have hand-crafted Libras translations that follow proper sign language grammar.
- Word lookup: For phrases not in the dictionary, the system looks up individual words in a sign dictionary of approximately 20 common signs (oi, obrigado, sim, nao, etc.).
- Grammar rules: As a final fallback, the system applies basic Libras grammar rules: articles and prepositions are removed, conjugated verbs are simplified to infinitive form, and negation is moved to the end of the sentence.

When Libras mode is active, each sign in the translated text is displayed as a colored rectangular card below the speech bubble text, with an 8-color palette cycling across signs for visual distinction. Cards automatically scale to fit the available bubble width.

7.4 Random Speech

The pet periodically speaks random phrases from its character model's phrase library. This creates a more lifelike and engaging companion experience. The interval between random speeches can be configured as either a fixed interval or a random range. The phrase selection uses a keyword-matching system that considers the conversation context to pick appropriate topics.

8 User Interface

The user interface consists of three main components: the floating pet window, the chat dialog, and the context menu. Each is designed to be unobtrusive while providing full access to all features.

8.1 Main Pet Window

The pet resides in a frameless, transparent GTK window that floats above other applications. The window has no title bar or borders, showing only the character sprite and speech bubble on a transparent background. Key interactions:

- Click and drag to move the pet anywhere on the screen. The position is saved to configuration.
- Double-click to open the chat window.
- Right-click to open the context menu with all configuration options.
- The window can be set to always stay on top of other windows.
- The character scale can be adjusted from 2x to 6x (default: 5x).

8.2 Speech Bubbles

Speech bubbles are rendered with Cairo and Pango directly on the pet window. They feature:

- Rounded rectangle background with semi-transparent white fill and subtle border.
- Animated tail pointing toward the character, positioned on the left or right side.
- Text positioning: auto (tail points away from screen center), left, or right.
- Automatic text wrapping with word-wrap mode for long messages.
- Minimum bubble width of 60 pixels; width expands to fit content up to the configured maximum.
- Duration-based display: each speech bubble stays visible for its configured duration, then fades or clears.

8.3 Chat Interface

The chat window provides a full messaging interface:

- Message history displayed as bubbles, with user messages on one side and pet responses on the other.
- Text entry field with send button for keyboard input.
- Microphone button for push-to-talk voice input (hold or toggle modes).
- Conversation history is saved per model to ~/.config/mate-helper/history/ and persists across sessions.
- Clear history button to reset the conversation.
- Emits speech signals to the main window, so chat responses appear as both text and speech bubbles.

8.4 Context Menu

The right-click context menu is organized into logical groups:

Menu Section	Settings
Chat	Open or close the chat window
Intelligence	AI provider selection, model selection for Ollama
Permissions	Seven individual toggles for AI tool access

Profile	User name and biography (injected into AI prompts)
Pet Model	Character model and language selection
Alarms	Add, list, toggle, and delete alarms
Audio and TTS	TTS enable/disable, provider selection, device, Fish Audio setup
Microphone (STT)	Microphone enable/disable, device selection, mode (hold/toggle)
Automation	Screen reading and audio transcription intervals and modes, Libras toggle
Shortcuts	Configure push-to-talk shortcut, view global shortcut help
Language	UI language selection (Portuguese, English, Japanese)
Window	Scale adjustment, always-on-top toggle, speech bubble side

8.5 Push-to-Talk Shortcut

The push-to-talk feature uses a Unix socket server running in a background thread. When the configured global shortcut is pressed, the application sends a signal to the socket, which triggers microphone recording. Recording continues until the key is released (in hold mode) or until a second press or timeout (in toggle mode). The shortcut can be configured from the context menu and works globally across all applications.

9 Configuration Reference

This chapter documents all user-configurable settings and their effects. Settings are persisted in ~/.config/mate-helper/config.json and can be modified through the context menu or by directly editing the JSON file.

9.1 AI Settings

Setting	Options	Description
AI Provider	Auto / Ollama / Groq / Gemini / HuggingFace / Phrases	Which AI service to use for conversations
Ollama Model	Any installed Ollama model name	Specific model to use when Ollama provider is active
Groq API Key	String	API key for Groq Cloud access
Gemini API Key	String	API key for Google Gemini access
HuggingFace Token	String	API token for HuggingFace Inference API

9.2 Audio Settings

Setting	Options	Description
TTS Enabled	On / Off	Enable or disable all text-to-speech output
TTS Provider	Auto / Fish Audio / Edge TTS / pyttsx3	Which TTS engine to use
TTS Device	PulseAudio sink name	Specific audio output device for TTS
Fish API Key	String	API key for Fish Audio TTS service
Fish Voice ID	String (optional)	Override the character's default Fish Audio voice
Mic STT Enabled	On / Off	Enable microphone speech-to-text
Mic Device	PulseAudio source name	Specific microphone device
Mic Mode	Hold / Toggle	Microphone activation mode
Mic Shortcut Key	Key name	Global keyboard shortcut for push-to-talk

9.3 Automation Settings

Setting	Options	Description
Screen Reading	Off / Exact / Random	Enable automatic screen description
Screen Interval	Seconds (exact) or min-max range	How often to capture and describe the screen
Audio Transcription	Off / Exact / Random	Enable automatic desktop audio transcription
Audio Interval	Seconds (exact) or min-max range	How often to capture and transcribe audio
Random Speech	On / Off	Enable random speech from phrase library
Speech Interval	Seconds (exact) or min-max range	How often the pet speaks unprompted
Libras Translation	On / Off	Enable written Libras translation in speech bubbles

9.4 Permissions System

Seven individual permission toggles control which tools the AI can execute:

- Read files - allows the AI to read any text file on the system
- List files - allows directory content listing
- Write files - allows creating and modifying files
- Run commands - allows bash command execution (with safety filters)
- Open URLs - allows opening websites in the browser

- Take screenshots - allows screen capture and analysis
- Listen to audio - allows desktop audio capture and transcription

NOTE
For security reasons, it is recommended to only enable the permissions you need. The "Run commands" permission in particular should be used with caution, as it gives the AI the ability to execute arbitrary bash commands.

9.5 Alarm System

Alarms are stored as part of the configuration file. Each alarm has:

- Time: Hour and minute for the alarm to trigger
- Name: A descriptive label shown when the alarm rings
- Enabled: Toggle to activate or deactivate without deleting
- Ringtone: The character model's configured MP3 ringtone file

When an alarm triggers, the pet switches to dancing mode, plays the ringtone, and displays an alarm message. Alarms can be stopped by clicking the alarm in the menu, typing stop words in chat ("para", "cala", "stop", "shut up"), or the alarm auto-stops after 30 seconds.

10 Custom Model Creation

One of Mate Helper's most powerful features is its extensible character model system. Anyone can create a new character by providing sprites, a personality definition, a phrase library, and UI translations. This chapter provides a complete guide to creating your own custom character model.

10.1 Understanding the Model System

Each character model is a Python package (a folder with an __init__.py file) placed in the models/ directory. The model system discovers available models by scanning for folders containing a model.py file. Models can be placed either in models/default_models/{language}/ for language-specific variants or directly in models/ for standalone custom models.

When the application starts or the model is changed via the menu, the model proxy loads the selected model's configuration, sprites, phrases, and strings. All of these are accessed through a singleton proxy object that caches the loaded module and reloads it when the model or language changes.

10.2 Model Directory Structure

A custom model requires the following file structure:

```
my_model/
  __init__.py       (empty file, marks as Python package)
  model.py          (identity, prompts, sprites config, TTS voices)
  phrases.py        (fallback phrases, keyword matching, alarms)
  strings.py        (UI translations for pt, en, jp)
  font.ttf          (optional custom font for speech bubbles)
  ringtone.mp3      (optional alarm ringtone)
  sprites/
    Default.png     (idle/normal pose)
    DefaultSpeaking.png  (speaking variant of normal)
    Happy.png
    HappySpeaking.png
    Sad.png
    SadSpeaking.png
    Angry.png
    AngrySpeaking.png
    Dancing.png or Dancing.gif  (celebration animation)
```

10.3 Sprites

Sprites are 32-pixel-wide images representing the character in different moods. Each mood requires a base sprite and an optional "speaking" variant with the mouth open. Sprites can be:

- PNG sprite sheets: A single PNG file containing frames arranged horizontally. The system auto-detects frame boundaries by scanning alpha-channel transitions. Each frame should be 32 pixels wide.
- Animated GIFs: Supported for the dancing mood. Pillow extracts individual frames and their timing, then renders them as an animation at 8 FPS.

For pixel-art characters, a scale of 5x (160 pixels on screen) provides a crisp retro look. The scale is configurable from the context menu. Sprites should be saved with transparency for proper rendering on

the transparent window background.

10.4 Model Configuration (model.py)

The model.py file defines your character's identity, personality, visual configuration, and voice settings. Key configuration fields include:

Field	Purpose	Example
MODEL_ID	Unique identifier for the model folder	my_custom_pet
PET_NAME	Full display name shown in UI	My Custom Pet
PET_SHORT_NAME	Short name used in chat	Pet
SPRITE_NAMES	Maps moods to sprite filenames	Normal -> Default, Feliz -> Happy
FONT_NAME	Font family for speech bubbles	Pixelify Sans
FONT_SIZE	Font size in points	13
SYSTEM_PROMPT	AI personality definition	"You are a cheerful companion..."
TTS_VOICE	Voice IDs for each TTS provider	{edge_tts: pt-BR-Voice}
RINGTONE_PATH	Path to alarm ringtone	ringtone.mp3

10.5 Personality Prompt Design

The SYSTEM_PROMPT is the single most important element of a character model. It defines how the AI behaves, speaks, and interacts. Effective prompts include:

- Character identity: name, background, and role (e.g., "You are a UTAUloid vocaloid")
- Speech patterns: desired tone, length, and style (e.g., "Respond in 1-2 short sentences with emotes")
- Behavioral rules: constraints and preferences (e.g., "Never introduce yourself as an AI")
- Language: which language to respond in (e.g., "Always respond in Brazilian Portuguese")
- Examples: one or two example exchanges showing desired response style

10.6 Phrase Library (phrases.py)

The phrase library provides fallback responses when no AI provider is available. It consists of:

- Categorized phrases: greeting, how_are_you, return_good, return_bad, thanks, bye, name, what_can_you_do, affection, jokes, sing, food, fun, curious, thanks_sarcastic, sleepy, learn, music, and unknown categories.
- Keyword matching: A mapping of keywords to categories, enabling context-aware response selection based on user input.
- Special phrases: Alarm notifications, thinking prefixes, conversation continuations, and tool result messages (screenshot taken, audio captured, file saved, etc.).
- A pick() function that returns a random phrase from a named category.

10.7 UI Strings (strings.py)

The strings module provides translations for approximately 160 UI labels, messages, and dialog texts. Each model ships with translations for Portuguese, English, and Japanese. Required string keys cover:

- Menu labels: All context menu item names and submenu titles
- Dialog texts: Configuration dialog descriptions and instructions
- Button labels: Cancel, Save, Close, Ok
- Status messages: Notifications for model loading, audio errors, configuration changes
- Speech keys: menu_libras, menu_fish_setup, tts_provider_*

A complete list of all required string keys is available in the custom model template at docs/custom_model/strings.py, which includes both the pt and en translations as a reference.

10.8 Testing Your Model

To test a custom model:

1. Copy your model folder to the desktop_pet/models/ directory.
2. Ensure your sprites are in the sprites/ subfolder with correct filenames.
3. Launch Mate Helper and select your model from the context menu (Pet Model -> your model name).
4. Test each mood by triggering different interactions (chat normally for happy, insult for sad, etc.).
5. Verify the speech bubble renders correctly with your custom font (if configured).
6. Test the AI personality by asking questions in the chat window.
7. Verify that fallback phrases work by disconnecting from the internet or disabling AI providers.

NOTE
A complete, commented custom model template is available in the docs/custom_model/ directory. Copy this folder as a starting point and modify the files to create your own character. The template includes explanatory comments for every field.

11 Troubleshooting

11.1 Common Issues

Issue	Likely Cause	Solution
Pet window not appearing	Missing GTK/Python dependencies	Install PyGObject, Pycairo, and GTK 3 development packag
No sprites visible	Missing sprite files or wrong paths	Verify sprites/ directory contains all required PNG/GIF files
AI not responding	Missing or invalid API key, no internet	Configure API key in Intelligence menu, or switch to Ollama
Microphone not working	Wrong device selected, PulseAudio not	Select correct mic source in Microphone menu, verify pactl
TTS silent	Missing provider dependencies, wrong device	Install edge-tts, check PulseAudio sink selection
Libras not translating	Feature not enabled	Enable in Automation menu -> Traducao para Libras
Alarms not ringing	Ringtone file missing	Verify ringtone.mp3 exists in model directory
Ollama connection failed	Ollama not installed or not running	Install Ollama, start with ollama serve, or choose a differer
Push-to-talk not working	pynput not installed, shortcut conflict	Install pynput, choose a different shortcut key

11.2 Logs and Debugging

The application outputs timestamped log messages to stderr. When running from a terminal, these messages provide visibility into what the application is doing:

- STT log entries show microphone capture status and transcribed text
- AI provider logs show which provider is being used and any errors
- TTS logs show which provider is active and audio playback status
- Timer logs show periodic task execution (screen reading, audio transcription)

Log messages use the format [HH:MM:SS] message. To capture logs for debugging, run the application with stderr redirected to a file:

```
python3 desktop_pet/main.py 2> mate-helper.log
```

11.3 Network and API Issues

Cloud AI providers and TTS services require internet access. If you experience connectivity issues:

- Switch to the Ollama provider for fully local AI inference
- Enable the "Phrases" provider to use only built-in fallback phrases
- For TTS, select pyttsx3 (offline) as the provider if available
- Check that your API keys are entered correctly and have not expired
- Verify that your firewall allows outbound HTTPS connections to the respective API endpoints

The auto-fallback mechanism means that even if your primary provider fails, the system will try alternative providers before resorting to offline mode.

11.4 Configuration File Recovery

If the configuration file becomes corrupted, delete ~/.config/mate-helper/config.json and restart the application. A fresh configuration file with defaults will be created automatically. Note that this will erase all settings, including API keys, alarms, and profile information.

Appendix A Configuration File Reference

The configuration file is stored at ~/.config/mate-helper/config.json. The following table documents all configuration keys and their default values.

Key	Type	Default	Purpose
window_x	int	centered	Pet window X position
window_y	int	centered	Pet window Y position
window_scale	int	5	Character sprite scale (2-6)
language	str	pt	UI language: pt, en, jp
active_model	str	kasane_teto	Active character model ID
provider	str	auto	AI provider selection
ai_enabled	bool	True	Master AI enable/disable
ollama_model	str	None	Ollama model preference
groq_key	str		Groq Cloud API key
gemini_key	str		Google Gemini API key
hf_token	str		HuggingFace API token
user_name	str		User's name for AI context
user_bio	str		User's biography for AI context
tool_read_file	bool	False	AI read file permission
tool_list_files	bool	False	AI list files permission
tool_write_file	bool	False	AI write file permission
tool_run_command	bool	False	AI run command permission
tool_open_url	bool	False	AI open URL permission
tool_screenshot	bool	False	AI screenshot permission
tool_listen	bool	False	AI audio capture permission
always_on_top	bool	True	Window always on top
bubble_side	str	auto	Speech bubble side
libras_enabled	bool	False	Libras translation toggle
tts_enabled	bool	False	TTS enable/disable
tts_provider	str	auto	TTS provider selection
tts_device	str		TTS audio output device
fish_audio_key	str		Fish Audio API key
fish_audio_voice	str		Fish Audio voice override
alarms	list	[]	List of alarm objects
mic_stt_enabled	bool	False	Microphone STT enable
mic_stt_device	str		Microphone device
mic_stt_mode	str	hold	Mic mode: hold or toggle
stt_shortcut_key	str		Push-to-talk shortcut key

Appendix B Glossary

Cairo
A 2D graphics library used for rendering the character sprite, speech bubbles, and Libras cards on the GTK drawing area.

Edge TTS
Microsoft's cloud-based text-to-speech service accessed through the edge-tts Python library. Provides high-quality voices without requiring an API key.

Fish Audio
A cloud TTS service offering high-quality neural voice synthesis. Requires an API key and supports custom voice profiles.

Gloss
In the context of Libras, a written representation of sign language using capitalized Portuguese words to represent signs (e.g., "OLA VOCE COMO-ESTA").

Groq
A cloud AI inference provider offering extremely low-latency access to Llama and Whisper models through custom hardware.

Libras
Lingua Brasileira de Sinais - Brazilian Sign Language, the official sign language of Brazil.

Model
A character definition package containing identity, sprites, personality prompt, phrases, UI strings, and TTS voice configuration.

Ollama
A local AI inference engine that runs large language models on the user's own machine without requiring internet access.

Pango
A text layout engine used for rendering speech bubble text with word wrapping, custom fonts, and international character support.

PulseAudio
The Linux sound server used for capturing microphone input, desktop audio, and managing TTS audio output.

pyttsx3
An offline text-to-speech library that wraps espeak on Linux. Provides TTS without internet access, though with lower voice quality.

STT
Speech-to-Text - the process of converting spoken audio into text using automatic speech recognition (Whisper).

TTS
Text-to-Speech - the process of converting written text into spoken audio using voice synthesis.

VAD
Voice Activity Detection - an algorithm that determines whether a segment of audio contains human speech, used to filter out background noise.

Whisper
OpenAI's automatic speech recognition model, accessed via Groq's API, used for transcribing microphone and desktop audio.